ISTQB® Certified Tester

Foundation Level Specialist — **Performance Testing** (CT-PT, Syllabus 2018)

CHAPTER 1
Basic Concepts

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	1.1 Principles of Performance Testing · 1.2 Types of Performance Testing · 1.3 Testing Types in Performance Testing · 1.4 The Concept of Load Generation · 1.5 Common Performance Efficiency Failure Modes and Their Causes
Learning objectives	5 examinable (PTFL-1.1.1 – PTFL-1.5.1)
Recommended teaching time	60 minutes — **6 marks per teaching hour, the best return in the syllabus**
Questions in the exam	**6 of 40 questions · 6 of 40 points.** K1 = 1 · **K2 = 5** · K3 = 0 · K4 = 0. Every objective is guaranteed; **PTFL-1.2 gets two questions**
Exam format	40 questions · 40 points · **90 minutes** (113 with the 25% non-native-language allowance) · pass mark 65% = **26 points**
Keywords	capacity testing · concurrency testing · efficiency · endurance testing · load generation · load testing · performance testing · scalability testing · spike testing · stress testing
Official sample-exam questions	**#1 – #6**, in learning-objective order

Built exclusively from the ISTQB® CT-PT 2018 syllabus, the official Sample Exam (questions and answers), and the ISTQB® exam structure tables.

Study this single file to master Chapter 1.

Contents

The page numbers above are filled in by Word: right-click the table of contents and choose "Update Field" → "Update entire table".

How to use this guide

Section 1 explains all five learning objectives in syllabus order — read it once, slowly. **Section 2** compresses the same material into recall-ready notes. **Section 3** holds the tables you should be able to reproduce from memory; **table 3.1 (the eight types)** and **table 3.3 (static versus dynamic)** are the two that pay directly. **Section 4** is the question bank — attempt every question with the green answer box covered, then read the rationale even when you were right. **Section 5** converts that into exam tactics. **Section 6** is for daily spaced repetition. **Section 7** is what you read on the morning of the exam.

What makes Chapter 1 unusual: it is the **cheapest chapter on the paper**. Six marks for **60 minutes** of recommended teaching gives **6 marks per teaching hour** — Chapter 4, by contrast, gives **2.5**. Everything here is **K1 and K2**: definition, recall and recognition. There is **no analysis and no calculation**, and **no selectable group**, so all five objectives appear on every paper.

Two facts that decide a third of the chapter: first, the official exam asks for the **best description** of a performance testing type **twice**, using an **identical set of four options** both times — **load, stress, scalability, spike**. Second, the K1 question maps a **dynamic activity to a test level**, and every distractor is a genuine activity placed one level away. Two tables, three marks.

Suggested schedule: Day 1 — Section 1 in full, then Section 3 tables 3.1 and 3.2. Day 2 — Section 2, plus rebuilding the eight-types table from blank paper. Day 3 — Section 4 Part A (all six official questions) and Part B for PTFL-1.1 and 1.2. Day 4 — Section 4 Part B for the remaining objectives, plus Section 5. Day 5 onwards — Section 6 flashcards daily, Section 7 the night before.

Section 1 — Syllabus Explanation

Chapter 1 carries **5 learning objectives** across five sub-chapters (1.1 Principles of Performance Testing · 1.2 Types of Performance Testing · 1.3 Testing Types in Performance Testing · 1.4 The Concept of Load Generation · 1.5 Common Performance Efficiency Failure Modes and Their Causes). The official ISTQB Exam Structure Table allocates **6 of the 40 exam questions** to Chapter 1 — **6 of the 40 points**. Recommended teaching time is **60 minutes**, which makes this the **highest points-per-hour chapter in the entire syllabus: 6 marks per teaching hour**.

Only one objective gets two questions — **PTFL-1.2 (types of performance testing)**. Every other objective in the chapter gets exactly one, so **all five objectives are guaranteed to appear**. There is no selectable group and nothing to gamble on: if you learn Chapter 1 completely you bank **6 of the 26 points you need to pass**, from one hour of teaching material.

LO ID	K	Objective (syllabus statement)	Qs	Pts
PTFL-1.1.1	K2	Understand the principles of performance testing.	**1**	1
PTFL-1.2.1	K2	Understand the different types of performance testing.	**2**	2
PTFL-1.3.1	K1	Recall testing types in performance testing.	**1**	1
PTFL-1.4.1	K2	Understand the concept of load generation.	**1**	1
PTFL-1.5.1	K2	Give examples of common failure modes of performance testing and their causes.	**1**	1
Chapter 1 total	—	**K1 = 1 · K2 = 5 · K3 = 0 · K4 = 0**	**6**	**6**

Exam allocation you must know

Chapter 1 = **6 questions / 6 points** of 40 questions and 40 points. In the official Sample Exam these are questions **#1 to #6**, in exact learning-objective order. The CT-PT exam is **40 questions worth 40 points (1 point per question) in 90 minutes** (**113 minutes** with the 25% non-native-language allowance) and the pass mark is **65% = 26 of 40 points**.

The one thing that decides this chapter

Two of the six marks — a third of the chapter — come from a single table: the eight types of performance testing. The official sample exam asks the same question twice with the same four options, once for spike testing and once for load testing. If you can map each of the eight types to its one-line syllabus definition, you have those 2 marks in under thirty seconds and you will almost certainly pick up the failure-mode mark too, because failure modes are described in the same vocabulary.

PTFL-1.1.1 (K2) — Principles of Performance Testing

Original syllabus statement

Understand the principles of performance testing.

Where performance efficiency sits. Technically, **performance efficiency is categorized in the ISO 25010 Product Quality Model as a non-functional quality characteristic**. That is the sentence to remember: it is **ISO 25010**, it is **product quality**, and it is **non-functional**. The syllabus writes *performance efficiency (or simply "performance")* — the two words are used interchangeably throughout.

The three sub-characteristics. Proper focus and prioritization among them **depends on the risks assessed and the needs of the various stakeholders**. Test results analysis may identify **other areas of risk that need to be addressed**.

Sub-characteristic	Syllabus definition	Trigger words in a stem
Time behavior	Generally the **most common performance testing objective**. Examines the ability of a component or system to **respond to user or system**	response time · seconds · end-to-end · CPU cycles · latency

Sub-characteristic	Syllabus definition	Trigger words in a stem
	inputs within a specified time and under specified conditions. Measurements may vary from the **"end-to-end" time** taken by the system to respond to user input, to the **number of CPU cycles** required by a software component to execute a particular task.	
Resource utilization	If the **availability of system resources is identified as a risk**, the utilization of those resources (e.g., **the allocation of limited RAM**) may be investigated by conducting specific performance tests.	RAM · CPU · disk · bandwidth · allocation
Capacity	If issues of **system behavior at the required capacity limits** of the system (e.g., **numbers of users or volumes of data**) are identified as a risk, performance tests may be conducted to **evaluate the suitability of the system architecture**.	number of users · data volume · limits · architecture suitability

The sentence that links this LO to Chapter 4

All three of the above quality sub-characteristics will impact the ability of the system under test (SUT) to scale. Scalability is not a fourth sub-characteristic — it is the consequence of the other three.

The five principles, in syllabus wording. The syllabus introduces them with "*the following performance testing principles are particularly relevant*". Learn them as five, in this order — the exam has asked which one is *an important principle*, and the correct answer was quoted almost verbatim.

#	Principle (syllabus wording)	What the exam does with it
1	Tests must be **aligned to the defined expectations of different stakeholder groups**, in particular **users, system designers and operations staff**.	The three named groups reappear in **PTFL-4.1.1** (user-based vs technical objectives) and **PTFL-4.1.3** (business vs technology stakeholders).
2	The tests must be **reproducible**. **Statistically identical results (within a specified tolerance) must be obtained by repeating the tests on an unchanged system**.	**This is the one the official exam picked as "an important principle".** Note the two qualifiers: *statistically identical*, and *within a specified tolerance* — not bit-for-bit identical.
3	The tests must **yield results that are both understandable and can be readily compared to stakeholder expectations**.	Understandability is why results are **aggregated** (**PTFL-2.2.1**) and why **dashboards** are used (**PTFL-3.1.1**, **PTFL-4.4.1**).
4	The tests **can be conducted, where resources allow, either on complete or partial systems or test environments that are representative of the production system**.	Note the wording: **representative of** production — not production itself. Distractors that demand testing **in** production are wrong (see **PTFL-4.2.8**).
5	The tests must be **practically affordable and executable within the timeframe set by the project**.	Affordability is why **cost versus representativeness** must be communicated to business stakeholders (**PTFL-4.1.3**).

Performance testing as experimentation. The syllabus states that performance testing **often takes the form of experimentation, which enables measurement and analysis of specific system parameters to take place**. These may be conducted **iteratively in support of system analysis, design and implementation** to enable **architectural decisions** to be made and to **help shape stakeholder expectations**. This is the root of the iterative theme that runs through Chapters 3 and 4.

Scope of the discipline. Performance testing is **not limited to the web-based domain** where the end user is the focus. It is also relevant to different application domains with a variety of system architectures, such as **classic client-server, distributed and embedded**. It is **often closely integrated with other disciplines such as usability engineering and performance engineering**. Additionally, evaluation of **functional suitability, usability and other quality characteristics under conditions of load** may reveal **load-specific issues** which impact those characteristics.

Exam focus. One K2 question, officially phrased as "*Which of the following is an important principle in performance testing?*" with **reproducibility** as the answer. The three distractors are worth knowing individually: *tests should be easy to create and understand* is wrong **because tests are sometimes quite complex to create**; *tests should be executed in the production environment* is wrong because tests **generally should not be run in the actual production environment, but rather a representative equivalent**; and *the test results should match the stakeholders' expectations* is wrong because **the test results may show that the system does not meet expectations**.

The trap in principle 3

Principle 3 says results must be **comparable to** stakeholder expectations. A distractor will turn this into results must **match** stakeholder expectations. **A performance test whose results are forced to match expectations is not a test.** Comparable — yes. Matching — no.

Related concepts: performance efficiency, ISO 25010 Product Quality Model, non-functional quality characteristic, time behavior, resource utilization, capacity, scalability, reproducibility, tolerance, representative environment, experimentation, performance engineering, usability engineering, system under test (SUT).

PTFL-1.2.1 (K2) — The Eight Types of Performance Testing

Original syllabus statement

Understand the different types of performance testing.

This objective is worth 2 of the 40 marks on the paper — the joint-largest single-objective allocation outside Chapter 4. The syllabus defines eight terms. One (**performance testing**) is the umbrella; the other seven are specific types. Every definition below is the syllabus's own, compressed to the phrase the exam reuses.

Type	Syllabus definition — learn the italicised phrase	The one word that gives it away
Performance testing	An **umbrella term** including **any kind of testing focused on performance (responsiveness) of the system or component under different volumes of load**.	*umbrella*
Load testing	Focuses on the ability of a system to handle **increasing levels of anticipated realistic loads** resulting from transaction requests generated by **controlled numbers of concurrent users or processes**.	*anticipated / expected maximum*
Stress testing	Focuses on the ability of a system or component to handle **peak loads that are at or beyond the limits of its anticipated or specified workloads**. Also used to evaluate a system's ability to handle **reduced availability of resources** such as accessible computing capacity, available bandwidth, and memory.	*at or beyond*
Scalability testing	Focuses on the ability of a system to meet **future efficiency requirements** which may be **beyond those currently required**. Determines the system's **ability to grow** (e.g., more users, larger amounts of data stored) **without violating the currently specified performance requirements or failing**. Once the limits of scalability are known, **threshold values can be set and monitored in production** to warn of problems about to arise; the production environment may then be **adjusted with appropriate amounts of hardware**.	*future*
Spike testing	Focuses on the ability of a system to **respond correctly to sudden bursts of peak loads and return afterwards to a steady state**.	*sudden burst · return to steady state*
Endurance testing	Focuses on the **stability of the system over a time frame specific to the system's operational context**. Verifies that there are **no resource capacity problems** (e.g., **memory leaks, database connections, thread pools**) that may **eventually degrade performance and/or cause failures at breaking points**.	*over time*

Type	Syllabus definition — learn the italicised phrase	The one word that gives it away
Concurrency testing	Focuses on the impact of situations where **specific actions occur simultaneously** (e.g., when **large numbers of users log in at the same time**). Concurrency issues are **notoriously difficult to find and reproduce**, particularly when the problem occurs in an environment where testing has **little or no control, such as production**.	*simultaneously / at the same time*
Capacity testing	Determines **how many users and/or transactions a given system will support and still meet the stated performance objectives**. These objectives may also be stated with regard to the **data volumes resulting from the transactions**.	*how many · and still meet*

The four definitions the official exam rotates

The sample exam asks about **spike** and **load** using an **identical set of four options**: (a) *loads gradually increased to reach the expected maximum* = **load**; (b) *loads at or beyond the expected peak load* = **stress**; (c) *meet future efficiency requirements* = **scalability**; (d) *respond to quick and extreme changes in load* = **spike**. Memorise this quartet as a block. Two of your six Chapter 1 marks live inside it.

Spike vs stress vs concurrency — the three that blur

Stress is about the **magnitude** of the load (at or beyond the limit) — and, uniquely, also about **reduced availability of resources**. **Spike** is about the **rate of change** of the load (a sudden burst) **and** about **returning to a steady state afterwards**. **Concurrency** is about **simultaneity of specific actions** — many users doing the **same thing at the same instant**, such as logging in. A stem that says *"beyond the expected peak"* is stress; *"sudden burst, then recovers"* is spike; *"all at the same time"* is concurrency.

Scalability vs capacity — the pair that blur

Capacity testing answers "how many?" for the system as it is today — how many users and/or transactions will it support **and still meet the stated performance objectives**. **Scalability testing answers "can it grow?"** — can it meet **future efficiency requirements beyond those currently required** without violating the **currently specified** performance requirements. Capacity measures the ceiling. Scalability asks whether the ceiling can be raised.

Exam focus. Two K2 questions, both of the form "*Which of the following is the best description of X testing?*". They are pure definition-matching. There is no scenario and no analysis — which is why this objective is the cheapest 2 marks on the paper.

Common misunderstanding. Treating **load testing** as "any test with load on it". Load testing specifically targets **anticipated, realistic** load — the load the system is **expected** to see, ramped up to the **expected maximum**. The moment the stem says the load exceeds what is expected, the answer becomes **stress**.

Related concepts: performance testing (umbrella), load testing, stress testing, scalability testing, spike testing, endurance testing, concurrency testing, capacity testing, anticipated load, peak load, steady state, memory leak, thread pool, threshold value.

PTFL-1.3.1 (K1) — Static and Dynamic Testing Types

Original syllabus statement

Recall testing types in performance testing.

This is one of only two K1 objectives on the whole paper (the other is PTFL-2.4.1). K1 means **remember or recognize** — you are not asked to apply anything, only to know which activity belongs where. The syllabus names exactly two testing types: **static testing** and **dynamic testing**.

Why static testing matters more here than elsewhere. The syllabus is blunt: **static testing activities are often more important for performance testing than for functional suitability testing**. The reason is that **so many critical performance defects are introduced in the architecture and design of the system**. Those defects can be introduced by **misunderstandings or a lack of knowledge by the designers and architects**, or because the **requirements did not adequately capture the response time, throughput, or resource utilization targets, the expected load and usage of the system, or the constraints**.

The four static testing activities — memorise the list

#	Static testing activity for performance
1	**Reviews of requirements** with focus on **performance aspects and risks**
2	**Reviews of database schemas, entity-relationship diagrams, metadata, stored procedures and queries**
3	**Reviews of the system and network architecture**
4	**Reviews of critical segments of the system code** (e.g., **complex algorithms**)

Dynamic testing across the test levels. "*As the system is built, dynamic performance testing should start as soon as possible.*" The syllabus lists an opportunity at **every one of the five test levels** — and the exam has asked directly which activity belongs at which level. This table is the answer key.

Test level	Dynamic performance testing opportunity (syllabus wording)
Unit testing	Using **profiling information to determine potential bottlenecks** and **dynamic analysis to evaluate resource utilization**.
Component integration testing	Across **key use cases and workflows**, especially when **integrating different use case features** or **integrating with the "backbone" structure of a workflow**.
System testing	**Overall end-to-end behaviors under various load conditions.**
System integration testing	Especially for **data flows and workflows across key inter-system interfaces**. Here it is **not uncommon for the "user" to be another system or machine** (e.g., inputs from sensors and other systems).
Acceptance testing	To **build user, customer and operator confidence** in the proper performance of the system and to **fine tune the system under real world conditions** — but **generally not to find performance defects in the system**.

> **Acceptance testing — the qualifier the exam loves**
>
> Acceptance testing is **generally not** for finding performance defects. Its purpose is **confidence building and fine tuning**. Any option that has you hunting for defects at acceptance level is wrong for this LO.

Two extra sentences that are examinable. First: in **higher test levels such as system testing and system integration testing, the use of realistic environments, data and loads are critical for accurate results**. Second: in **Agile and other iterative-incremental lifecycles, teams should incorporate static and dynamic performance testing into early iterations rather than waiting for final iterations** to address performance risks.

Custom or new hardware. If custom or new hardware is part of the system, **early dynamic performance tests can be performed using simulators**. However, it is **good practice to start testing on the actual hardware as soon as possible**, because **simulators often do not adequately capture resource constraints and performance-related behaviors**.

Exam focus. One K1 question. The official version asks which performance testing activity should occur **during unit testing** — the answer is **testing to evaluate resource utilization and potential bottlenecks**. The distractors are all real activities placed at

the wrong level: *end-to-end behavior under load* belongs to **system testing**, *data flows and workflows across interfaces* belongs to **system integration testing**, and *key use cases and workflows* belongs to **component integration testing**. The whole question is decided by the table above.

Related concepts: static testing, dynamic testing, review, database schema, entity-relationship diagram, stored procedure, profiling, dynamic analysis, bottleneck, unit testing, component integration testing, system testing, system integration testing, acceptance testing, simulator, iterative-incremental lifecycle.

PTFL-1.4.1 (K2) — The Concept of Load Generation

Original syllabus statement

Understand the concept of load generation.

Why load is not just test data. In order to carry out the types of performance testing in Section 1.2, **representative system loads must be modeled, generated and submitted to the system under test**. Loads are **comparable to the data inputs used for functional test cases**, but differ in **three principal ways** — and the exam expects all three.

#	How a performance test load differs from functional test data
1	A performance test load must represent **many user inputs, not just one**.
2	A performance test load may require **dedicated hardware and tools** for generation.
3	Generation of a performance test load is **dependent on the absence of any functional defects in the system under test** which may impact test execution.

The success factor sentence

The efficient and reliable generation of a specified load is a key success factor when conducting performance tests. That single sentence is the reason this learning objective exists, and the reason Chapter 5 (Tools) exists.

The four options for load generation. The syllabus gives four, each with its own trade-off. The exam question for this LO is a **selection** question: given a situation, which generation approach fits?

Option	When it is adequate	Its limitations
Via the user interface	When **only a small number of users** are to be represented **and** the required numbers of **software clients are available**. May be used **in conjunction with functional test execution tools**. **May be the most representative approach for end-to-end tests.**	**Rapidly becomes impractical as the number of users to be simulated increases.** The **stability of the UI is a critical dependency** — **frequent changes can impact the repeatability** of performance tests and **may significantly affect maintenance costs**.
Using crowds	When a **large number of testers** are available to represent real users, organised so the desired load is generated. Suitable for **applications reachable from anywhere in the world** (e.g., web-based), and may involve **a wide range of different device types and configurations**.	The load generated will **not be as reproducible and precise as other options** and is **more complex to organize**.
Via the API	Similar to using the UI for data entry, but uses the **application's API instead of the UI**. **Less sensitive to changes (e.g., delays) in the UI** and allows **transactions to be processed in the same way as they would if entered directly by a user via the UI**. **Dedicated scripts may repeatedly call specific API**	Bypasses the UI itself, so genuine UI-layer effects are not exercised.

Option	When it is adequate	Its limitations
	routines, enabling **more users to be simulated compared to UI inputs**.	
Using captured communication protocols	Capturing user interaction with the SUT **at the communications protocol level** and then **replaying these scripts** to simulate **potentially very large numbers of users in a repeatable and reliable manner**. This tool-based approach is described in **Sections 4.2.6 and 4.2.7**.	Requires **correlation and parameterization** of the recorded script (see PTFL-4.2.7) — the main reason recorded scripts fail to play back.

API vs protocol level — the distinction that decides the mark

The official rationale spells it out: **the communications protocol level is one level below the API level.** If a stem says *"the UI is likely to change but the transactions must be processed as if they were created through the UI"*, the answer is **API**. If the stem emphasises **capture and replay** or **very large numbers of users, repeatably**, the answer is **captured communication protocols**. And a stem about **a large number of testers representing real users** is **crowd testing**, not API — because **those testers would be using the real UI**.

Exam focus. One K2 question: "*When is it appropriate to generate load via the application's APIs?*" Answer: **when the UI is likely to change but the transactions must be processed as if they were created through the UI**. The distractor "*when only small numbers of test instances are available*" is wrong precisely because **many test instances can be created with API testing** — the API route is the scalable one relative to the UI.

Related concepts: load generation, virtual user, user interface, crowd testing, application programming interface (API), communication protocol, capture and replay, reproducibility, repeatability, maintenance cost, functional defect dependency.

PTFL-1.5.1 (K2) — Common Failure Modes and Their Causes

Original syllabus statement

Give examples of common failure modes of performance testing and their causes.

The four failure modes. The syllabus names four, each with its own list of underlying causes. The exam question for this LO works **backwards**: you are given a cause (a memory leak) and asked what you will observe. Learn this table in both directions.

Failure mode (what you observe)	Syllabus description	Underlying causes
Slow response under all load levels	**Response is unacceptable regardless of load.**	**Bad database design or implementation · network latency · other background loads.** Such issues **can be identified during functional and usability testing, not just performance testing**, so test analysts should keep an eye open for them and report them.
Slow response under moderate-to-heavy load levels	Response **degrades unacceptably with moderate-to-heavy load, even when such loads are entirely within normal, expected, allowed ranges**.	**Saturation of one or more resources · varying background loads.**
Degraded response over time	Response **degrades gradually or severely over time.**	**Memory leaks · disk fragmentation · increasing network load over time · growth of the file repository · unexpected database growth.**
Inadequate or graceless error handling under heavy	**Response time is acceptable but error handling degrades** at **high and beyond-limit load levels**.	**Insufficient resource pools · undersized queues and stacks · too rapid time-out settings.**

Failure mode (what you observe)	Syllabus description	Underlying causes
or over-limit load		

> **The four-way discrimination the exam is built on**
>
> **Regardless of load → underlying design/latency/background load. Only under heavy load → resource saturation. Worsens over time → memory leak, fragmentation, repository or database growth. Response fine but errors go wrong → pools, queues, stacks, timeouts.** In the official question, a **memory leak** produces **response time degrading over time** — the answer is decided entirely by which column the cause sits in.

The six specific examples. The syllabus then gives six concrete failures, each closing with a diagnosis sentence. The diagnosis sentence is the examinable part, because it names the sub-characteristic at fault.

Specific example	Syllabus diagnosis
A **web-based application** providing information about a company's services **does not respond to user requests within seven seconds** (a general industry rule of thumb).	**The performance efficiency of the system cannot be achieved under specific load conditions.**
A system **crashes or is unable to respond** when subjected to a **sudden large number of user requests** (e.g., ticket sales for a major sporting event).	**The capacity of the system to handle this number of users is inadequate.**
System response is **significantly degraded** when users request **large amounts of data** (e.g., a large and important report posted for download).	**The capacity of the system to handle the generated data volumes is insufficient.**
Batch processing is unable to complete before online processing is needed.	**The execution time of the batch processes is insufficient for the time period allowed.**
A **real-time system runs out of RAM** when parallel processes generate large demands for dynamic memory which **cannot be released in time**.	**The RAM is not dimensioned adequately, or requests for RAM are not adequately prioritized.**
Real-time component **A**, supplying inputs to component **B**, is **unable to calculate updates at the required rate**; the overall system fails to respond in time and may fail.	**Code modules in component A must be evaluated and modified ("performance profiling") to ensure the required update rates can be achieved.**

Exam focus. One K2 question: "*If you have an application that has a memory leak, what is the likely result you will see during performance testing?*" Answer: **response time will degrade over time** — with the official caveat that **the time length is unknown until you test it**. Every distractor is another row of the failure-mode table, which is why learning the table answers the question by elimination as well as directly.

Common misunderstanding. Assuming that *slow under heavy load* means the system is broken. The syllabus is precise: this failure mode is defined as degradation **even when such loads are entirely within normal, expected, allowed ranges**. Slowness at loads **beyond** the specified range is a different matter and belongs to **stress testing**.

Related concepts: failure mode, memory leak, resource saturation, disk fragmentation, background load, network latency, resource pool, queue, stack, time-out setting, batch processing window, performance profiling, dimensioning, seven-second rule of thumb.

Section 2 — Key Concepts

Concise, exam-ready notes drawn from the syllabus text. Every item is traceable to Chapter 1 of the CT-PT syllabus (pages 10–16).

2.1 Performance efficiency

- **Performance efficiency (or simply "performance")** is categorized in the **ISO 25010 Product Quality Model** as a **non-functional quality characteristic**.
- It has **three sub-characteristics**: **time behavior**, **resource utilization** and **capacity**.
- **Focus and prioritization among them depends on the risks assessed and the needs of the various stakeholders.** Test results analysis may identify **other areas of risk**.
- **All three sub-characteristics impact the ability of the SUT to scale.**
- Performance testing **plays a critical role in establishing acceptable quality levels for the end user** and is **often closely integrated with usability engineering and performance engineering**.
- Evaluating **functional suitability, usability and other quality characteristics under conditions of load** may reveal **load-specific issues** impacting those characteristics.
- Performance testing is **not limited to the web-based domain**: it also applies to **classic client-server, distributed and embedded** architectures.
- Performance testing **often takes the form of experimentation**, conducted **iteratively in support of system analysis, design and implementation**, to enable **architectural decisions** and to **shape stakeholder expectations**.

2.2 The five principles

1. **Aligned to the defined expectations of different stakeholder groups** — in particular **users, system designers and operations staff**.
2. **Reproducible** — **statistically identical results within a specified tolerance** when repeated **on an unchanged system**.
3. **Understandable and readily comparable to stakeholder expectations.**
4. **Conducted, where resources allow, on complete or partial systems or test environments representative of the production system.**
5. **Practically affordable and executable within the timeframe set by the project.**

2.3 The eight types in one line each

- **Performance testing** — the **umbrella term** for any testing focused on **performance (responsiveness) under different volumes of load**.
- **Load testing** — **increasing levels of anticipated realistic loads** from **controlled numbers of concurrent users or processes**.
- **Stress testing** — **peak loads at or beyond the limits** of anticipated or specified workloads; also **reduced availability of resources** (computing capacity, bandwidth, memory).
- **Scalability testing** — the ability to meet **future efficiency requirements beyond those currently required**, growing **without violating the currently specified performance requirements or failing**.
- **Spike testing** — **sudden bursts of peak loads**, and **returning afterwards to a steady state**.
- **Endurance testing** — **stability over a time frame specific to the system's operational context**; catches **memory leaks, database connections, thread pools**.
- **Concurrency testing** — the impact of **specific actions occurring simultaneously**; issues are **notoriously difficult to find and reproduce**.
- **Capacity testing** — **how many users and/or transactions** the system will support **and still meet the stated performance objectives**.

2.4 Static and dynamic testing

- **Static testing is often more important for performance testing than for functional suitability testing**, because **so many critical performance defects are introduced in the architecture and design**.
- Root causes: **misunderstandings or lack of knowledge by designers and architects**, or **requirements that did not adequately capture response time, throughput or resource utilization targets, expected load and usage, or constraints**.
- **Four static activities:** reviews of **requirements** (performance aspects and risks) · **database schemas, ERDs, metadata, stored procedures and queries** · **system and network architecture** · **critical segments of the system code** (e.g., complex algorithms).
- **Dynamic performance testing should start as soon as possible** as the system is built.
- **Unit** → profiling for bottlenecks, dynamic analysis of resource utilization. **Component integration** → key use cases and workflows, feature integration, the workflow backbone. **System** → end-to-end behaviors under various load. **System integration** → data flows and workflows across inter-system interfaces, where the "user" may be **another system or machine**. **Acceptance** → **confidence and fine tuning**, generally **not** defect-finding.
- In **higher test levels**, **realistic environments, data and loads are critical for accurate results**.
- In **Agile and other iterative-incremental lifecycles**, incorporate **static and dynamic performance testing into early iterations**.
- **Custom or new hardware:** simulators enable early dynamic tests, but **start on the actual hardware as soon as possible** — **simulators often do not adequately capture resource constraints and performance-related behaviors**.

2.5 Load generation

- Loads must be **modeled, generated and submitted** to the SUT.
- **Three differences from functional test data:** many user inputs not just one · may need **dedicated hardware and tools** · depends on the **absence of functional defects** in the SUT.
- **The efficient and reliable generation of a specified load is a key success factor.**
- **Via the UI** — small numbers of users, clients available; may pair with **functional test execution tools**; **most representative for end-to-end**; but **impractical at scale** and **UI stability is a critical dependency** affecting **repeatability and maintenance cost**.
- **Using crowds** — real testers organised to produce the load; good for **globally reachable applications** and **many device types**; but **less reproducible and precise**, and **more complex to organize**.
- **Via the API** — **less sensitive to UI changes**, transactions **processed as if entered via the UI**, **dedicated scripts call API routines**, **more users simulated than via UI**.
- **Using captured communication protocols** — record at the **protocol level**, replay to simulate **very large numbers of users repeatably and reliably**; the tool-based approach of **Sections 4.2.6 and 4.2.7**.
- **The protocol level is one level below the API level.**

2.6 Failure modes

- **Slow response under all load levels** → bad **database design or implementation**, **network latency**, **other background loads.** Findable in **functional and usability testing** too.
- **Slow response under moderate-to-heavy load levels** → **saturation of one or more resources**, **varying background loads** — and note it happens **within normal, expected, allowed ranges**.
- **Degraded response over time** → **memory leaks**, **disk fragmentation**, **increasing network load over time**, **growth of the file repository**, **unexpected database growth**.
- **Inadequate or graceless error handling under heavy or over-limit load** → **insufficient resource pools**, **undersized queues and stacks**, **too rapid time-out settings**.
- Specific examples map to a diagnosis: **seven-second web response** → performance efficiency not achieved under specific load; **crash on sudden large numbers of requests** → **capacity for users inadequate**; **degradation on large data requests** → **capacity for data volumes insufficient**; **batch overruns its window** → **execution time insufficient for the**

time period allowed; **real-time system out of RAM → RAM not dimensioned adequately or requests not prioritized**; **component A too slow for component B → performance profiling and modification of code modules in A**.

2.7 Chapter 1 keyword glossary

The syllabus lists **ten keywords** for Chapter 1. All parts of the syllabus are examinable at **K1**, so every keyword below must be recognisable by definition even where no learning objective names it.

Keyword	Definition to memorise
Capacity testing	Determines **how many users and/or transactions a given system will support and still meet the stated performance objectives**; objectives may also cover **data volumes resulting from the transactions**.
Concurrency testing	Focuses on the impact of situations where **specific actions occur simultaneously**, e.g., large numbers of users logging in at the same time. Issues are **notoriously difficult to find and reproduce**.
Efficiency	Short for **performance efficiency** — the **ISO 25010 non-functional product quality characteristic** comprising **time behavior, resource utilization and capacity**.
Endurance testing	Focuses on the **stability of the system over a time frame specific to the system's operational context**, verifying there are **no resource capacity problems** that may eventually **degrade performance or cause failures at breaking points**.
Load generation	The **modeling, generation and submission of representative system loads** to the SUT. Four options: **UI · crowds · API · captured communication protocols**.
Load testing	Focuses on the ability of a system to handle **increasing levels of anticipated realistic loads** from **controlled numbers of concurrent users or processes**.
Performance testing	**Umbrella term** for **any kind of testing focused on performance (responsiveness) of the system or component under different volumes of load**.
Scalability testing	Focuses on the ability of a system to meet **future efficiency requirements** which may be **beyond those currently required**, without **violating the currently specified performance requirements or failing**.
Spike testing	Focuses on the ability of a system to **respond correctly to sudden bursts of peak loads and return afterwards to a steady state**.
Stress testing	Focuses on the ability to handle **peak loads at or beyond the limits of anticipated or specified workloads**, and to handle **reduced availability of resources**.

Section 3 — Tables and Summaries

3.1 The eight types — the table that is worth 2 marks

Type	Load shape	Question it answers	Signature phrase
Performance testing	Any	Is the system responsive under load?	**umbrella term · responsiveness · different volumes of load**
Load testing	Ramping up to the expected maximum	Does it cope with the load we expect?	**increasing levels of anticipated realistic loads**
Stress testing	At or beyond the limit; or resources removed	What happens past the limit?	**at or beyond the limits · reduced availability of resources**
Scalability testing	Growing beyond today's requirement	Can it grow?	**future efficiency requirements · beyond those currently required**
Spike testing	A sudden burst, then back down	Does it survive a shock and recover?	**sudden bursts of peak loads · return afterwards to a steady state**
Endurance testing	Constant, for a long time	Does it stay stable?	**stability over a time frame · no resource capacity problems**
Concurrency testing	Many identical actions at one instant	What breaks when everyone acts at once?	**specific actions occur simultaneously**
Capacity testing	Increasing until the objective breaks	How many, exactly?	**how many users and/or transactions · and still meet the stated performance objectives**

The four-option block used by the official exam

(a) loads **gradually increased to reach the expected maximum → LOAD**. **(b)** loads **at or beyond the expected peak load → STRESS**. **(c) meet future efficiency requirements → SCALABILITY**. **(d) respond to quick and extreme changes in load → SPIKE**. Learn these four as one unit and both PTFL-1.2 marks are automatic.

3.2 Reverse lookup — symptom to type

A stem that says...	...is asking about
"gradually increased", "expected maximum", "anticipated realistic"	**Load testing**
"at or beyond", "exceeds the specified workload", "reduced bandwidth or memory"	**Stress testing**
"future", "grow", "more users later", "larger amounts of data stored"	**Scalability testing**
"sudden burst", "quick and extreme change", "returns to a steady state", "transient state"	**Spike testing**
"over a long period", "memory leak", "thread pool", "gradual degradation", "breaking point"	**Endurance testing**
"at the same time", "simultaneously", "all log in at once", "hard to reproduce"	**Concurrency testing**
"how many users", "how many transactions", "and still meet the objectives"	**Capacity testing**

3.3 Static versus dynamic — the K1 answer key

Activity	Type	Level
Review of **requirements** for **performance aspects and risks**	**Static**	Any / early
Review of **database schemas, ERDs, metadata, stored procedures, queries**	**Static**	Any / early
Review of the **system and network architecture**	**Static**	Any / early
Review of **critical segments of the system code** (complex algorithms)	**Static**	Any / early
Profiling for potential bottlenecks; **dynamic analysis** of resource utilization	**Dynamic**	**Unit testing**
Key use cases and workflows; integrating features or the workflow **backbone**	**Dynamic**	**Component integration testing**
End-to-end behaviors under various load conditions	**Dynamic**	**System testing**
Data flows and workflows across key inter-system interfaces (the "user" may be another system)	**Dynamic**	**System integration testing**
Building user, customer and operator confidence; **fine tuning under real world conditions**	**Dynamic**	**Acceptance testing**

3.4 Load generation — the selection grid

Approach	Choose it when	Reject it when
User interface	Few users · clients available · **end-to-end representativeness** matters · pairing with **functional test execution tools**	Many users needed · the **UI changes frequently** (repeatability and maintenance cost)
Crowds	**Many real testers** available · application **reachable from anywhere** · **many device types and configurations** wanted	**Reproducibility and precision** are required · organising effort is a constraint
API	**The UI is likely to change** but transactions must be processed **as if entered via the UI** · more users needed than the UI allows	The test must exercise the **UI layer itself**
Captured communication protocols	**Very large numbers of users**, **repeatable and reliable** · tool-based recording and replay	Correlation/parameterization effort is unaffordable (see PTFL-4.2.7)

3.5 Failure modes — cause to symptom, and back

If the cause is...	...the symptom is	Failure mode
Bad **database design or implementation**; **network latency**; **other background loads**	Unacceptable response **at every load level**	**Slow response under all load levels**
Saturation of one or more resources; **varying background loads**	Response degrades **within normal, expected ranges**	**Slow response under moderate-to-heavy load**
Memory leaks; **disk fragmentation**; **increasing network load**; **file repository growth**; **unexpected database growth**	Response degrades **gradually or severely over time**	**Degraded response over time**
Insufficient resource pools; **undersized queues and stacks**; **too rapid time-out settings**	Response time **acceptable** but **error handling degrades**	**Inadequate or graceless error handling**

3.6 One-line answer to every learning objective

LO	If you remember only one sentence
PTFL-1.1.1	Performance efficiency is an **ISO 25010 non-functional characteristic** with **time behavior, resource utilization and capacity**; of the five principles the exam picks **reproducibility — statistically identical results within a specified tolerance on an unchanged system**.
PTFL-1.2.1	Eight types; the exam rotates four — **load = anticipated maximum · stress = at or beyond · scalability = future · spike = sudden burst then steady state**.
PTFL-1.3.1	**Static** = four kinds of review, and it matters **more** here than in functional testing because defects come from **architecture and design**. **Dynamic** = one opportunity per test level, and **unit testing means profiling and resource utilization**.
PTFL-1.4.1	Four ways to generate load; **API when the UI will change but transactions must behave as if entered via the UI**, and the **protocol level is one level below the API level**.
PTFL-1.5.1	Four failure modes discriminated by **when** they appear: **always · only under load · over time · only in the error handling** — a **memory leak** is always **over time**.

Section 4 — Demo Questions with Answers

*Questions 1–6 are the **official ISTQB CT-PT Sample Exam questions for Chapter 1** — questions #1 to #6, reproduced with their official correct answers and rationale. Questions 7–34 are additional items derived strictly from the syllabus text, in the same style and difficulty. Correct options are marked ✔ — cover the green box and self-test first.*

Part A — The 6 official Chapter 1 questions

Q1. OFFICIAL · Sample Exam #1

Which of the following is an important principle in performance testing?

a) The tests should be easy to create and understand

b) The test results must be reproducible when the system under test is unchanged ✔

c) The tests should be executed in the production environment to provide the most accurate results

d) The test results should match the stakeholders' expectations for system performance

Correct answer: b | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: (b) Reproducible results is a key factor in performance test execution. The results should be statistically identical when run on the same unchanged system. (a) is not correct **because tests are sometimes quite complex to create**. (c) is not correct because tests **generally should not be run in the actual production environment, but rather a representative equivalent**. (d) is not correct because **the test results may show that the system does not meet expectations** — the principle is that results are **comparable to** expectations, not that they match them.

Q2. OFFICIAL · Sample Exam #2

Which of the following is the best description of spike testing?

a) It focuses on the ability of the system to handle loads that are gradually increased to reach the expected maximum

b) It focuses on the ability of the system to handle loads that are at or beyond the expected peak load

c) It focuses on the ability of the system to meet future efficiency requirements

d) It focuses on the ability of the system to respond to quick and extreme changes in load ✔

Correct answer: d | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: (d) is the description of spike testing — the ability to **respond correctly to sudden bursts of peak loads and return afterwards to a steady state**. **(a) is a description of load testing. (b) is a description of stress testing. (c) is a description of scalability testing.** These four options are the block the exam reuses; learn them together.

Q3. OFFICIAL · Sample Exam #3

Which of the following is the best description of load testing?

a) It focuses on the ability of the system to handle loads that are gradually increased to reach the expected maximum ✔

b) It focuses on the ability of the system to handle loads that are at or beyond the expected peak load

c) It focuses on the ability of the system to meet future efficiency requirements

d) It focuses on the ability of the system to respond to quick and extreme changes in load

Correct answer: a | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: (a) is the description of load testing — the ability to handle **increasing levels of anticipated realistic loads** resulting from transaction requests generated by **controlled numbers of concurrent users or processes**. **(b) is a description of stress testing. (c) is a description of scalability testing. (d) is a description of spike testing.** Note that the official exam asked the identical question twice with identical options, once for spike and once for load — **2 of your 6 chapter marks from one table**.

Q4. OFFICIAL · Sample Exam #4

Which of the following performance testing activities should occur during unit testing?

a) Testing end-to-end behavior under various load conditions

b) Testing data flows and workflows across interfaces

c) Testing key use cases and workflows using a top-down approach

d) Testing to evaluate resource utilization and potential bottlenecks ✔

Correct answer: d | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.2

Why: (d) While this can occur at any time, it should start with unit testing — the syllabus lists **profiling information to determine potential bottlenecks and dynamic analysis to evaluate resource utilization** as the unit-level opportunity. (a) is not correct because this occurs at **system or system integration testing**. (b) is not correct because this occurs during **system integration testing**. (c) is not correct because this is done during **component integration testing**.

Q5. OFFICIAL · Sample Exam #5

When is it appropriate to generate load via the application's APIs?

a) When a large number of testers are available who can represent the real users

b) When testing must be conducted at the communications protocol level

c) When the UI is likely to change but the transactions must be processed as if they were created through the UI ✔

d) When only small numbers of test instances are available

Correct answer: c | LO: PTFL-1.4.1 (K2) | Syllabus §1.4

Why: (c) Testing through the API makes the most sense when the UI is likely to change, but the full communication to and from the UI must be tested. The API approach is **less sensitive to changes (e.g., delays) in the UI** and allows **transactions to be processed in the same way as they would if entered directly by a user via the UI**. (a) is not correct because **this is crowd testing and these testers would be using the real UI**. (b) is not correct because **the communications protocol level is one level below the API level**. (d) is not correct because **many test instances can be created with API testing**.

Q6. OFFICIAL · Sample Exam #6

If you have an application that has a memory leak, what is the likely result you will see during performance testing?

a) Response time will be consistently slow

b) Response time will remain acceptable but the error handling will degrade

c) Response time will be slow, but only under heavy loads

d) Response time will degrade over time ✔

Correct answer: d | LO: PTFL-1.5.1 (K2) | Syllabus §1.5

Why: (d) With a memory leak, response time will degrade over time — the time length is **unknown until you test it**. (a) is an indication of **underlying performance issues such as bad design, under-powered hardware, network latency**. (b) is an indication of **resource pool, queue or stack issues as well as invalid timeout settings**. (c) is an indication that **some of the resources may become saturated, but only at high loads**. All four options are the four failure modes — the question is decided entirely by which one owns *memory leak*.

Part B — 28 additional syllabus-derived questions

Q7. Recall

In which standard's product quality model is performance efficiency categorized, and as what kind of characteristic?

a) ISO/IEC 7498-1, as a communications characteristic

b) ISO 25010, as a non-functional quality characteristic ✔

c) ISO 26262, as a safety-related characteristic

d) ISO 25010, as a functional quality characteristic

Correct answer: b | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: The syllabus states that **performance efficiency is categorized in the ISO 25010 Product Quality Model as a non-functional quality characteristic** with **three sub-characteristics**. ISO/IEC 7498-1 is the **OSI model** reference used in §4.2.1 for communication protocols — a favourite distractor because both standards appear in this syllabus.

Q8. Definition matching

Which sub-characteristic of performance efficiency is described as "generally the most common performance testing objective"?

a) Capacity

b) Resource utilization

c) Time behavior ✔

d) Scalability

Correct answer: c | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: Time behavior — "*generally the evaluation of time behavior is the most common performance testing objective*". It examines the ability to **respond to user or system inputs within a specified time and under specified conditions**, and its measurements range from **end-to-end time** down to **the number of CPU cycles** a component needs. **Scalability is not a sub-characteristic at all** — the syllabus says **all three sub-characteristics impact the ability of the SUT to scale**.

Q9. Classification

A risk analysis has flagged that the system may not behave correctly when the number of stored records grows to the levels the business expects, and the team wants to know whether the chosen architecture is suitable.

Which sub-characteristic of performance efficiency is being investigated?

a) Capacity ✔

b) Time behavior

c) Resource utilization

d) Reproducibility

Correct answer: a | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: Capacity. The syllabus defines it as: if issues of **system behavior at the required capacity limits** of the system (e.g., **numbers of users or volumes of data**) are identified as a risk, performance tests may be conducted to **evaluate the suitability of the system architecture**. The phrase *suitability of the system architecture* appears only under capacity. **Reproducibility is a principle, not a sub-characteristic.**

Q10. Tricky

Which statement about the scope of performance testing is consistent with the syllabus?

a) Performance testing applies only to web-based systems, where the end user is the focus

b) Performance testing is a purely technical discipline and is kept separate from usability engineering

c) Testing other quality characteristics under load rarely reveals anything new about those characteristics

d) Performance testing applies to a variety of architectures including classic client-server, distributed and embedded, and is often closely integrated with usability engineering and performance engineering ✔

Correct answer: d | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: The syllabus states performance testing is **not limited to the web-based domain**; it is **also relevant to different application domains with a variety of system architectures, such as classic client-server, distributed and embedded**, and is **often closely integrated with other disciplines such as usability engineering and performance engineering**. It also states that evaluating **functional suitability, usability and other quality characteristics under conditions of load may reveal load-specific issues** — which rules out (c).

Q11. Tricky · principle wording

The reproducibility principle requires that repeating a test on an unchanged system yields which of the following?

a) Bit-for-bit identical results

b) Statistically identical results within a specified tolerance ✔

c) Results that improve on each successive run as caches warm up

d) Results identical to the production environment's measured behaviour

Correct answer: b | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: The syllabus wording is precise: **statistically identical results (within a specified tolerance) must be obtained by repeating the tests on an unchanged system**. Both qualifiers matter. Option (c) describes the **caching problem** the syllabus warns about in §4.2.7, which is a defect in the test, not a property of a reproducible one.

Q12. Recall · list

The first performance testing principle requires that tests be aligned to the defined expectations of different stakeholder groups. Which three groups does the syllabus name in particular?

a) Testers, developers and auditors

b) Customers, suppliers and regulators

c) Users, system designers and operations staff ✔

d) Product owners, scrum masters and architects

Correct answer: c | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: Users, system designers and operations staff. These three reappear later: the **user-based versus technical objectives** split in **PTFL-4.1.1**, and the **business-focused versus technology-focused stakeholder communication** in **PTFL-4.1.3**. Learning them once pays three times.

Q13. Tricky · environment

According to the performance testing principles, where may performance tests be conducted?

a) Only on the complete production system, since anything smaller is unrepresentative

b) On complete or partial systems, or test environments that are representative of the production system, where resources allow ✔

c) Only in an isolated laboratory, never on partial systems

d) Anywhere, since the environment has no measurable effect on performance results

Correct answer: b | LO: PTFL-1.1.1 (K2) | Syllabus §1.1

Why: Principle 4: the tests **can be conducted, where resources allow, either on complete or partial systems or test environments that are representative of the production system**. Note the three permitted targets and the qualifier **where resources allow**. Option (d) contradicts **PTFL-4.2.8**, which states that **performance is a non-linear function of the environment**.

Q14. Definition matching

Which of the following is the best description of stress testing?

a) It focuses on the ability of the system to handle loads that are gradually increased to reach the expected maximum

b) It focuses on the stability of the system over a time frame specific to its operational context

c) It focuses on the ability to handle peak loads at or beyond the limits of anticipated workloads, and on reduced availability of resources ✔

d) It determines how many users a given system will support while still meeting the stated performance objectives

Correct answer: c | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Stress testing focuses on peak loads **at or beyond the limits of anticipated or specified workloads**, and is **also used to evaluate a system's ability to handle reduced availability of resources such as accessible computing capacity, available bandwidth, and memory**. That second clause is unique to stress testing and is the fastest way to recognise it. (a) is load, (b) is endurance, (d) is capacity.

Q15. Definition matching

Which type of performance testing verifies that there are no resource capacity problems — such as memory leaks, database connections or thread pools — that may eventually degrade performance or cause failures at breaking points?

a) Endurance testing ✔

b) Stress testing

c) Concurrency testing

d) Capacity testing

Correct answer: a | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Endurance testing focuses on the **stability of the system over a time frame specific to the system's operational context**, and the three examples of resource capacity problems the syllabus names are exactly **memory leaks, database connections and thread pools**. Notice how this dovetails with the **degraded response over time** failure mode in §1.5.

Q16. Tricky · pair

What distinguishes capacity testing from scalability testing?

a) Capacity testing uses virtual users; scalability testing uses real users

b) Capacity testing is static; scalability testing is dynamic

c) Capacity testing runs for longer than scalability testing

d) Capacity testing determines how many users or transactions the system supports while still meeting stated objectives; scalability testing addresses the ability to meet future efficiency requirements beyond those currently required ✔

Correct answer: d | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Capacity measures today's ceiling — *how many users and/or transactions a given system will support and still meet the stated performance objectives*. **Scalability asks whether the ceiling can be raised** — the ability to meet **future efficiency requirements which may be beyond those currently required**, growing **without violating the currently specified performance requirements or failing**. Neither is defined by tooling, duration, or static-versus-dynamic.

Q17. Application

An online examination platform expects every candidate in a national cohort to press "Start" at exactly 09:00. The team is worried about what happens at that instant rather than about the sustained load afterwards.

Which type of performance testing most directly addresses this concern?

a) Endurance testing

b) Concurrency testing ✔

c) Scalability testing

d) Capacity testing

Correct answer: b | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Concurrency testing focuses on the impact of situations where **specific actions occur simultaneously** — the syllabus's own example is **large numbers of users logging in at the same time**. The syllabus adds that such issues are **notoriously difficult to find and reproduce**, particularly in an environment where testing has **little or no control, such as production**.

Q18. Tricky

According to the syllabus, what becomes possible once the limits of scalability are known?

a) The system can be certified as defect-free at any load

b) Stress testing becomes unnecessary

c) Threshold values can be set and monitored in production to warn of problems about to arise, and the production environment may be adjusted with appropriate amounts of hardware ✔

d) The operational profile can be replaced by a fixed load profile

Correct answer: c | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: The syllabus states: "*Once the limits of scalability are known,* ***threshold values can be set and monitored in production to provide a warning of problems which may be about to arise****. In addition the production environment may be* ***adjusted with appropriate amounts of hardware****.*" This is one of the few places where the syllabus explicitly links a test type to **production monitoring** — and it connects to the **monitoring tools** discussion in **PTFL-5.1.1**.

Q19. Recall

Which term does the syllabus define as an umbrella term including any kind of testing focused on the responsiveness of the system or component under different volumes of load?

a) Performance testing ✔

b) Load testing

c) Efficiency testing

d) Capacity testing

Correct answer: a | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Performance testing is the umbrella. The distinction matters: a stem that describes a specific behaviour (anticipated maximum, sudden burst, long duration) is asking about a **specific type**, whereas a stem that describes the whole discipline is asking about the umbrella. **Load testing is not the umbrella** — it is one of the seven specific types under it.

Q20. Application

A payments gateway currently handles 400 transactions per second within its response-time targets. The business plans to acquire a competitor and expects volumes to triple within eighteen months. Management wants to know whether the current architecture can grow to meet that demand without breaching today's response-time requirements.

Which type of performance testing is being requested?

a) Capacity testing

b) Scalability testing ✔

c) Stress testing

d) Load testing

Correct answer: b | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Scalability testing — the objective is to determine the system's **ability to grow** (more users, larger data volumes) **without violating the currently specified performance requirements or failing**, in order to meet **future efficiency requirements which may be beyond those currently required**. Capacity testing would answer "how many can we take **today**"; scalability answers "can we take **more later**".

Q21. Tricky · elimination

A test drives the system to 130% of its specified maximum workload and then also repeats the run with half the normal memory available. Which single type covers both of these?

a) Spike testing

b) Capacity testing

c) Stress testing ✔

d) Concurrency testing

Correct answer: c | LO: PTFL-1.2.1 (K2) | Syllabus §1.2

Why: Stress testing is the only type whose definition covers **both** halves: peak loads **at or beyond the limits of anticipated or specified workloads**, **and** the ability to handle **reduced availability of resources such as accessible computing capacity, available bandwidth, and memory**. Spike testing is about a **sudden burst and the return to a steady state**, not about resource starvation.

Q22. Recall

Why does the syllabus say static testing activities are often more important for performance testing than for functional suitability testing?

a) Because dynamic performance testing is prohibited before system testing

b) Because static testing is cheaper than dynamic testing in every project

c) Because performance defects cannot be detected dynamically

d) Because so many critical performance defects are introduced in the architecture and design of the system ✔

Correct answer: d | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.1

Why: The syllabus reason is exactly (d). It then names the sources: **misunderstandings or a lack of knowledge by the designers and architects**, and **requirements that did not adequately capture the response time, throughput, or resource utilization targets, the expected load and usage of the system, or the constraints**. Cost is never given as the reason.

Q23. Recall · list

Which of the following is listed by the syllabus as a static testing activity for performance?

a) Profiling a component under load to find bottlenecks

b) Reviews of database schemas, entity-relationship diagrams, metadata, stored procedures and queries ✔

c) Executing end-to-end workflows with realistic data volumes

d) Running a scaled-down load test against a partial system

Correct answer: b | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.1

Why: The four static activities are: reviews of **requirements** (performance aspects and risks) · reviews of **database schemas, entity-relationship diagrams, metadata, stored procedures and queries** · reviews of the **system and network architecture** · reviews of **critical segments of the system code** (e.g., complex algorithms). Options (a), (c) and (d) all involve **execution**, so they are dynamic by definition.

Q24. Level mapping

At which test level does the syllabus note that it is not uncommon for the "user" to be another system or machine, for example inputs from sensors?

a) Unit testing

b) Component integration testing

c) System integration testing ✔

d) Acceptance testing

Correct answer: c | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.2

Why: System integration testing, where dynamic performance testing focuses on **data flows and workflows across key inter-system interfaces**. The syllabus explicitly notes that at this level the "user" is often **another system or machine**. This idea returns in **PTFL-4.2.3** as the *systems of systems* operational profile.

Q25. Tricky · qualifier

What is the stated purpose of dynamic performance testing during acceptance testing?

a) To build user, customer and operator confidence and to fine tune the system under real world conditions — but generally not to find performance defects ✔

b) To find as many performance defects as possible before release

c) To replace system-level performance testing when time is short

d) To establish the baseline for the first time

Correct answer: a | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.2

Why: The syllabus qualifier is explicit: acceptance-level dynamic performance testing exists to **build user, customer and operator confidence** and to **fine tune the system under real world conditions**, and is **generally not to find performance defects in the system**. Any option that turns acceptance testing into a defect hunt is wrong for this LO.

Q26. Recall

What does the syllabus advise about using simulators when custom or new hardware is part of the system?

a) Simulators should be used for the whole project because real hardware is too expensive

b) Simulators must never be used for performance work

c) Simulators capture resource constraints more accurately than the real hardware

d) Early dynamic performance tests can use simulators, but it is good practice to start testing on the actual hardware as soon as possible ✔

Correct answer: d | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.2

Why: The syllabus reason for moving to real hardware is that **simulators often do not adequately capture resource constraints and performance-related behaviors**. Simulators are permitted for **early** tests only — they are a bridge, not a destination.

Q27. Lifecycle

What does the syllabus recommend for Agile and other iterative-incremental lifecycles?

a) Defer all performance testing to a hardening iteration at the end

b) Incorporate static and dynamic performance testing into early iterations rather than waiting for final iterations ✔

c) Perform only static performance testing, since iterations are too short for dynamic tests

d) Perform performance testing only when a dedicated performance team is available

Correct answer: b | LO: PTFL-1.3.1 (K1) | Syllabus §1.3.2

Why: "*In Agile and other iterative-incremental lifecycles, teams should incorporate **static and dynamic performance testing into early iterations** rather than waiting for final iterations to address performance risks.*" This is the Chapter 1 seed of the much longer treatment in **PTFL-3.4.1**, where dedicated performance iterations and CI performance tests are discussed.

Q28. Recall · list

Which of the following is one of the three ways in which a performance test load differs from the data inputs used for functional test cases?

a) It must always be generated from the production environment

b) It must be recorded rather than programmed

c) Its generation is dependent on the absence of any functional defects in the system under test which may impact test execution ✔

d) It must always be reproducible to the millisecond

Correct answer: c | LO: PTFL-1.4.1 (K2) | Syllabus §1.4

Why: The three differences are: the load must represent **many user inputs, not just one**; it **may require dedicated hardware and tools** for generation; and its generation is **dependent on the absence of any functional defects in the SUT** which may impact test execution. That third one is the least intuitive and therefore the most examinable — you cannot load-test through a functional defect.

Q29. Application

A consumer web application must be exercised from many countries, on a wide range of real device types and configurations, and the organisation has access to a large pool of testers willing to take part.

Which load generation approach is being described, and what is its principal limitation?

a) Crowd-based load generation; the load will not be as reproducible and precise as other options and is more complex to organize ✔

b) API-based load generation; it bypasses the UI layer

c) Protocol-level capture and replay; correlation is difficult

d) UI-based load generation; the UI may change frequently

Correct answer: a | LO: PTFL-1.4.1 (K2) | Syllabus §1.4

Why: This is **load generation using crowds**: testers organised so the **desired load can be generated**, suitable for applications **reachable from anywhere in the world** and involving **a wide range of different device types and configurations**. Its stated downside is that the load is **not as reproducible and precise as other options** and it is **more complex to organize**.

Q30. Tricky

Which load generation approach does the syllabus describe as potentially the most representative for end-to-end tests, while also warning that it rapidly becomes impractical as the number of simulated users increases?

a) Load generation using captured communication protocols

b) Load generation via the user interface ✔

c) Load generation via the API

d) Load generation using crowds

Correct answer: b | LO: PTFL-1.4.1 (K2) | Syllabus §1.4

Why: Via the user interface. The syllabus says "*Testing through the UI may be the most representative approach for end-to-end tests*", but also that it **may rapidly become impractical as the numbers of users to be simulated increases**, and that the **stability of the UI represents a critical dependency — frequent changes can impact the repeatability of performance tests and may significantly affect the maintenance costs**.

Q31. Tricky · levels

How does the syllabus relate the API level to the communications protocol level?

a) They are the same level, described with two names

b) The API level is one level below the communications protocol level

c) The protocol level is a subset of the crowd testing approach

d) The communications protocol level is one level below the API level ✔

Correct answer: d | LO: PTFL-1.4.1 (K2) | Syllabus §1.4

Why: The official rationale for Sample Exam question #5 states it directly: **the communications protocol level is one level below the API level**. This ordering is what makes the protocol approach able to simulate **potentially very large numbers of users in a repeatable and reliable manner**, and what makes **correlation** necessary (see **PTFL-4.2.7**).

Q32. Cause to symptom

A system's response times remain acceptable at all load levels, but at beyond-limit loads its error handling becomes graceless and unhelpful. Which underlying defects does the syllabus associate with this failure mode?

a) Memory leaks, disk fragmentation and unexpected database growth

b) Insufficient resource pools, undersized queues and stacks, and too rapid time-out settings ✔

c) Bad database design, network latency and background loads

d) Saturation of one or more resources and varying background loads

Correct answer: b | LO: PTFL-1.5.1 (K2) | Syllabus §1.5

Why: The failure mode is **inadequate or graceless error handling under heavy or over-limit load**, and its underlying defects are exactly **insufficient resource pools, undersized queues and stacks, and too rapid time-out settings**. Option (a) belongs to **degraded response over time**, (c) to **slow response under all load levels**, and (d) to **slow response under moderate-to-heavy load levels**.

Q33. Tricky

Which failure mode does the syllabus say can be identified during functional and usability testing as well as during performance testing?

a) Degraded response over time

b) Inadequate error handling under over-limit load

c) Slow response under all load levels ✔

d) Slow response under moderate-to-heavy load levels

Correct answer: c | LO: PTFL-1.5.1 (K2) | Syllabus §1.5

Why: For **slow response under all load levels**, the syllabus notes that the underlying issues — **bad database design or implementation, network latency, and other background loads** — "*can be identified during functional and usability testing, not just performance testing, so* ***test analysts should keep an eye open for them and report them***". This is the only failure mode carrying that instruction, because it is the only one that shows up **regardless of load**.

Q34. Diagnosis

A batch reconciliation job that must finish before the online day begins is still running when the first users log in.

Which diagnosis does the syllabus give for this specific example?

a) The execution time of the batch processes is insufficient for the time period allowed ✔

b) The capacity of the system to handle this number of users is inadequate

c) The RAM is not dimensioned adequately

d) The performance efficiency of the system cannot be achieved under specific load conditions

Correct answer: a | LO: PTFL-1.5.1 (K2) | Syllabus §1.5

Why: The syllabus example reads: "*Batch processing is unable to complete before online processing is needed.* ***The execution time of the batch processes is insufficient for the time period allowed.***" Each of the six specific examples closes with a one-line diagnosis, and the exam can quote either half and ask for the other. Batch windows return in **PTFL-4.2.3** (off-line batch processing operational profiles) and **PTFL-4.2.5** (throughput within a nightly batch window).

Most Important Questions From This Chapter

*Ranking rationale: the ISTQB Exam Structure Table gives Chapter 1 six questions across five objectives, with **PTFL-1.2 receiving two**. Every objective is guaranteed, so the ranking reflects **marks at stake per unit of study effort**.*

Rank	Topic / question focus	LO	Why it matters
1	The eight types of performance testing, especially the load / stress / scalability / spike quartet — Q2, Q3, Q14–Q21	PTFL-1.2.1	**2 guaranteed marks**, asked as pure definition matching with an **identical option set** both times. The cheapest two marks on the entire paper.
2	Which dynamic activity belongs at which test level, plus the four static review activities — Q4, Q22–Q27	PTFL-1.3.1	**1 guaranteed mark** at **K1**, decided by a single table. Also underpins Chapter 3's lifecycle questions.
3	The four failure modes and their causes, in both directions — Q6, Q32–Q34	PTFL-1.5.1	**1 guaranteed mark.** All four distractors are the other three modes, so the table answers by elimination as well as directly.
4	The four load generation options and the API-versus-protocol distinction — Q5, Q28–Q31	PTFL-1.4.1	**1 guaranteed mark**, and the vocabulary feeds **PTFL-4.2.6** and **PTFL-4.2.7** in the 20-mark chapter.
5	The five principles and the three sub-characteristics — Q1, Q7–Q13	PTFL-1.1.1	**1 guaranteed mark**, and **reproducibility** is the answer the official exam chose. The stakeholder trio also reappears twice in Chapter 4.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

The ISTQB Exam Structure Table for CT-PT allocates Chapter 1 **six questions**. Only **PTFL-1.2** receives two; the rest receive one each. There is **no selectable group**, so **all five objectives appear on every paper**.

LO	K	Questions	Points	Sample Exam #	Format used officially
PTFL-1.1	K2	**1**	**1**	#1	"Which of the following is **an important principle**" — one true statement among three near-misses
PTFL-1.2	K2	**2**	**2**	#2, #3	"Which is the **best description** of X testing?" — asked twice with an **identical option set**
PTFL-1.3	**K1**	**1**	**1**	#4	"Which activity should occur **during unit testing**?" — level mapping
PTFL-1.4	K2	**1**	**1**	#5	"**When is it appropriate** to generate load via the API?" — situation selection
PTFL-1.5	K2	**1**	**1**	#6	Cause given, symptom asked — "what is the **likely result** you will see?"
Chapter 1 total	—	**6**	**6**	**#1–#6**	**15% of the paper, for 60 minutes of teaching — the best return per minute in the syllabus**

> **The arithmetic before you sit the exam**
>
> CT-PT is **40 questions worth 40 points (one point each) in 90 minutes** (**113 minutes** with the 25% non-native-language allowance); the pass mark is **65% = 26 points**. Chapter allocation: **Ch1 = 6 · Ch2 = 4 · Ch3 = 7 · Ch4 = 20 · Ch5 = 3**. Teaching time: **Ch1 = 60 min · Ch2 = 55 · Ch3 = 195 · Ch4 = 475 · Ch5 = 90**. Chapter 1 gives **6 marks per teaching hour**; Chapter 4 gives **2.5**. Chapters 1 and 2 together are **10 of the 26 points you need**, from **115 minutes** of material.

5.2 Common distractors observed

- **"Tests should be easy to create and understand"** offered as a principle — the syllabus says results must be **understandable**, and explicitly notes that **tests are sometimes quite complex to create**.
- **"Tests should be executed in the production environment for accuracy"** — tests should run in an environment **representative of** production. Only **PTFL-4.1.2** admits production as a last resort, **and only with the specific risks discussed**.
- **"Results should match stakeholder expectations"** — they must be **readily comparable to** expectations. A result that fails expectations is still a valid result.
- **Load and stress swapped.** *Gradually increased to the expected maximum* is **load**; *at or beyond the expected peak* is **stress**.
- **Spike and stress swapped.** Spike is about the **speed of change** and the **return to a steady state**; stress is about the **magnitude** and about **reduced resources**.
- **Scalability described as "current" capability** — scalability is always about **future efficiency requirements beyond those currently required**. "How many can we take today" is **capacity**.
- **Endurance described as "long test"** without the resource angle — endurance exists to catch **memory leaks, database connections and thread pools** that **degrade performance or cause failures at breaking points**.
- **Right activity, wrong test level.** Every distractor in the official K1 question is a genuine syllabus activity placed one level away from where it belongs.
- **Acceptance testing described as defect hunting** — it is **generally not** for finding performance defects.
- **Crowd testing described as API testing** — crowd testers **use the real UI**.

- **"Small numbers of test instances"** offered as a reason to use the API — the opposite is true: **many test instances can be created with API testing**.
- **Memory leak mapped to "consistently slow"** — a leak is always **degradation over time**.
- **"Slow only under heavy load"** offered for a memory leak — that is **resource saturation**, a different mode.

5.3 Keyword recognition (map words → answers)

Keyword in the stem	Points toward
'time behavior', 'respond within a specified time', 'CPU cycles', 'end-to-end time'	**Time behavior** sub-characteristic (PTFL-1.1.1)
'allocation of limited RAM', 'availability of system resources'	**Resource utilization** sub-characteristic (PTFL-1.1.1)
'numbers of users or volumes of data', 'capacity limits', 'suitability of the system architecture'	**Capacity** sub-characteristic (PTFL-1.1.1)
'statistically identical', 'within a specified tolerance', 'unchanged system'	**Reproducibility** principle (PTFL-1.1.1)
'users, system designers and operations staff'	The **stakeholder alignment** principle (PTFL-1.1.1)
'gradually increased', 'anticipated realistic loads', 'expected maximum'	**Load testing** (PTFL-1.2.1)
'at or beyond the limits', 'reduced availability of resources', 'bandwidth and memory'	**Stress testing** (PTFL-1.2.1)
'future efficiency requirements', 'grow', 'threshold values monitored in production'	**Scalability testing** (PTFL-1.2.1)
'sudden bursts', 'quick and extreme changes', 'return afterwards to a steady state'	**Spike testing** (PTFL-1.2.1)
'stability over a time frame', 'memory leaks, database connections, thread pools', 'breaking points'	**Endurance testing** (PTFL-1.2.1)
'occur simultaneously', 'log in at the same time', 'difficult to find and reproduce'	**Concurrency testing** (PTFL-1.2.1)
'how many users and/or transactions', 'and still meet the stated performance objectives'	**Capacity testing** (PTFL-1.2.1)
'reviews of', 'database schemas', 'entity-relationship diagrams', 'stored procedures', 'network architecture', 'complex algorithms'	**Static testing** (PTFL-1.3.1)
'profiling', 'dynamic analysis', 'resource utilization', 'potential bottlenecks'	Dynamic testing at **unit** level (PTFL-1.3.1)
'key use cases and workflows', 'backbone structure of a workflow'	Dynamic testing at **component integration** level (PTFL-1.3.1)
'end-to-end behaviors under various load conditions'	Dynamic testing at **system** level (PTFL-1.3.1)
'data flows and workflows across interfaces', 'the user is another system or machine', 'sensor inputs'	Dynamic testing at **system integration** level (PTFL-1.3.1)
'build confidence', 'fine tune under real world conditions'	Dynamic testing at **acceptance** level (PTFL-1.3.1)
'simulators', 'custom or new hardware'	Start early on simulators, **move to actual hardware as soon as possible** (PTFL-1.3.1)
'a large number of testers who represent real users'	**Crowd** load generation (PTFL-1.4.1)
'the UI is likely to change', 'processed as if entered via the UI'	**API** load generation (PTFL-1.4.1)
'capture at the communications protocol level', 'replay', 'very large	**Captured communication protocols** (PTFL-1.4.1)

Keyword in the stem	Points toward
numbers of users repeatably'	
'unacceptable regardless of load'	**Slow response under all load levels** — bad DB design, latency, background load (PTFL-1.5.1)
'degrades within normal expected ranges'	**Slow under moderate-to-heavy load** — resource saturation (PTFL-1.5.1)
'memory leak', 'disk fragmentation', 'unexpected database growth', 'file repository growth'	**Degraded response over time** (PTFL-1.5.1)
'error handling degrades', 'insufficient pools', 'undersized queues and stacks', 'too rapid time-outs'	**Inadequate or graceless error handling** (PTFL-1.5.1)

5.4 Typical traps

- **The same four options appear twice.** Sample Exam #2 and #3 share an identical option set. Do not assume the second question must have a different answer letter — read the type being asked about, not the option text you already recognise.
- **"Best description" means one is exactly right.** These are definition questions, not judgement calls. If two options feel close, one of them is the definition of a **different type on the same list**.
- **K1 does not mean easy.** PTFL-1.3.1 is K1, but the question requires you to hold **five test levels and their activities** in memory simultaneously. Recall is only easy if the table is memorised.
- **Watch the qualifier words in principles.** *Comparable to*, *representative of*, *statistically identical*, *within a specified tolerance*, *where resources allow*. Distractors are built by deleting or hardening exactly these qualifiers.
- **A cause can produce more than one plausible symptom in real life — but only one in the syllabus.** Answer from the table, not from field experience.
- **Scalability is not a sub-characteristic.** It is what all three sub-characteristics **impact**. An option that lists four sub-characteristics is wrong on its face.

5.5 How to eliminate wrong answers

- Strike any principle statement that requires testing **in** production, or that requires results to **match** expectations.
- Strike any type definition that mentions **the expected maximum** unless the question asks about **load**.
- Strike any type definition that mentions **future** unless the question asks about **scalability**.
- Strike any type definition mentioning a **return to steady state** unless the question asks about **spike**.
- Strike any level-mapping option that puts **end-to-end under load** anywhere but **system testing**, or **profiling** anywhere but **unit testing**.
- Strike any load-generation option that describes **real testers on the real UI** when the question asks about the **API**.
- Strike any failure-mode option whose **timing** does not match the cause: leaks are **over time**, saturation is **under load**, design and latency are **always**, pools and timeouts affect **error handling**.

5.6 Scenario-solving shortcuts

- **Type questions:** ask only two things — **how big is the load?** and **how fast does it change?** Big and steady → load or stress (expected vs beyond). Fast and sharp → spike. Simultaneous and identical → concurrency. Long → endurance. Counting → capacity. Future → scalability.
- **Level questions:** work bottom-up. Unit → **inside one component** (profiling). Component integration → **between components** (use cases, workflows). System → **the whole system end-to-end**. System integration → **between systems**. Acceptance → **confidence, not defects**.
- **Load generation questions:** ask **how many users** and **is the UI stable?** Few users and stable UI → UI. Many users and changing UI → API. Very many users, repeatable → protocol capture. Many real people, many devices → crowds.

- **Failure mode questions:** ask **when does it hurt?** Always · only when busy · progressively · only when things go wrong. Those four answers are the four modes.

5.7 Common candidate mistakes

- Skimming Chapter 1 because it is "just the basics". It is **6 of the 26 points you need**, for **60 minutes of teaching** — proportionally the richest chapter in the syllabus.
- Learning the eight types as prose instead of as a **table with one signature phrase each**. The exam gives you four definitions and asks which is which; prose does not survive that.
- Ignoring **concurrency testing** and **capacity testing** because they did not appear in the sample exam. Both are examinable and both are easy to confuse with their neighbours.
- Forgetting that **PTFL-1.2 is worth two marks**, and budgeting study time as if all five objectives were equal.
- Confusing **performance testing** (the umbrella) with **load testing** (one type under it).
- Treating the **failure modes** as engineering intuition rather than as a fixed four-row table with fixed causes.
- Missing that **static testing matters more here than in functional testing** — a fact the exam can ask about directly.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Unpacks to
T-R-C ("**TRaCk** performance")	The three ISO 25010 performance efficiency sub-characteristics: **T**ime behavior · **R**esource utilization · **C**apacity. And remember: **all three impact scalability**, which is **not** a fourth.
A-R-U-R-A ("**A RURAl** principle")	The five principles: **A**ligned to stakeholder expectations · **R**eproducible · **U**nderstandable and comparable · **R**epresentative environments (complete or partial) · **A**ffordable within the project timeframe.
"Expected · Beyond · Future · Sudden"	The four-option block the exam reuses: **load** = up to the expected maximum · **stress** = at or beyond · **scalability** = future · **spike** = sudden change.
"Long, Loud, Lots"	**Long** = endurance (over time) · **Loud** = concurrency (everyone at once) · **Lots** = capacity (how many, exactly).
"Review the four Rs"	Static testing = reviews of **R**equirements · **R**elational design (schemas, ERDs, metadata, stored procedures, queries) · **A**rchitecture (system and network) · **C**ode (critical segments, complex algorithms).
U-C-S-S-A ("**U C** the **S**ystem **S**tep by step to **A**cceptance")	The dynamic ladder: **U**nit (profiling, resource utilization) · **C**omponent integration (use cases, workflows, backbone) · **S**ystem (end-to-end under load) · **S**ystem integration (inter-system data flows) · **A**cceptance (confidence and fine tuning, not defects).
U-C-A-P ("**U CAP**ture the load")	The four load-generation options: **U**ser interface · **C**rowds · **A**PI · **P**rotocol capture. Ordered by increasing scale and decreasing UI realism.
"One below"	**The communications protocol level is one level below the API level.** This single sentence answers the API-versus-protocol distractor every time.
A-M-T-E ("**A M**emory **T**hat **E**rodes")	The four failure modes by timing: **A**lways slow · **M**oderate-to-heavy load slow · **T**ime-degraded · **E**rror handling fails. A memory leak is always the **T**.
"Leak = later"	Memory leak → **degraded response over time**. Never *consistently slow*, never *only under heavy load*.

5.9 Chapter 1 mind map

CHAPTER 1 — Basic Concepts · 60 minutes · 6 exam marks · K1 = 1, K2 = 5 · all objectives guaranteed

Branch	Node (K)	Everything hanging off it
1.1 Principles	**1.1.1** Principles of performance testing (K2) — **1 mark**	**Performance efficiency** = **ISO 25010 non-functional product quality characteristic** · **time behavior** (most common objective; end-to-end time down to **CPU cycles**) · **resource utilization** (e.g., **allocation of limited RAM**) · **capacity** (**numbers of users or volumes of data**; **suitability of the system architecture**) · **all three impact the ability to scale** · **five principles**: aligned to **users, system designers, operations staff** · **reproducible** (statistically identical within a specified tolerance, unchanged system) · **understandable and comparable** · **complete or partial systems or representative environments** · **affordable and within the project timeframe** · performance testing as **experimentation** shaping **architectural decisions** and **stakeholder expectations** · applies to **client-server, distributed, embedded**, not just web
1.2 Types	**1.2.1** Types of performance testing (K2) — **2 marks**	**Performance testing** = umbrella (**responsiveness under different volumes of load**) · **load** = **increasing levels of anticipated realistic loads**, controlled numbers of concurrent users or processes · **stress** = **at or beyond the limits**, plus **reduced availability of resources** · **scalability** = **future efficiency requirements beyond those currently required**, grow **without violating current requirements**, then **threshold values monitored in production** and **hardware adjusted** · **spike** = **sudden bursts** and **return to a steady state** · **endurance** = **stability over a time frame specific to the operational context**; **memory leaks, database connections, thread pools**; **breaking points** · **concurrency** = **specific actions occurring simultaneously**; **notoriously difficult to find and reproduce** · **capacity** = **how many users and/or transactions** while **still meeting the stated performance objectives**, including **data volumes**
1.3 Testing types	**1.3.1** Static and dynamic (**K1**) — **1 mark**	**Static matters more here than in functional testing** because **critical performance defects are introduced in architecture and design** — from **designer/architect misunderstanding** or **requirements that missed response time, throughput, resource utilization targets, expected load and usage, or constraints** · **four reviews**: requirements · **database schemas, ERDs, metadata, stored procedures, queries** · **system and network architecture** · **critical code segments (complex algorithms)** · **dynamic starts as soon as possible**: **unit** = profiling for bottlenecks + dynamic analysis of resource utilization · **component integration** = key use cases and workflows, feature integration, the **backbone** · **system** = **end-to-end under various load** · **system integration** = **inter-system data flows**, the "user" may be **another system or machine** · **acceptance** = **confidence and fine tuning**, **generally not defect finding** · higher levels need **realistic environments, data and loads** · **Agile: put static and dynamic into early iterations** · **simulators early, real hardware as soon as possible**
1.4 Load generation	**1.4.1** The concept of load generation (K2) — **1 mark**	Loads are **modeled, generated and submitted** · **three differences from functional data**: **many inputs not one** · **may need dedicated hardware and tools** · **depends on the absence of functional defects** · **efficient and reliable generation is a key success factor** · **UI** (few users, clients available, **most representative for end-to-end**, but **impractical at scale** and **UI stability is a critical dependency** affecting **repeatability and maintenance cost**) · **crowds** (real testers, **anywhere in the world**, **many device types**, but **less reproducible and precise**, **more complex to organize**) · **API** (**less sensitive to UI changes**, **processed as if entered via the UI**, **more users than UI**) · **captured protocols** (**very large numbers of users, repeatable and reliable**; see §4.2.6 and §4.2.7) · **the protocol level is one level below the API level**
1.5 Failure modes	**1.5.1** Common failure modes and causes (K2) — **1 mark**	**Slow under all load levels → bad database design or implementation, network latency, other background loads**; also findable in **functional and usability testing** · **slow under moderate-to-heavy load → saturation of one or more resources, varying background loads**, and it happens **within normal**

Branch	Node (K)	Everything hanging off it
		expected allowed ranges · degraded over time → memory leaks, disk fragmentation, increasing network load, file repository growth, unexpected database growth · inadequate or graceless error handling → insufficient resource pools, undersized queues and stacks, too rapid time-out settings · six specific examples with diagnoses: **seven-second web response · crash on sudden requests = capacity for users inadequate · large data requests = capacity for data volumes insufficient · batch overrun = execution time insufficient for the period allowed · real-time RAM exhaustion = RAM not dimensioned adequately or requests not prioritized · component A too slow for B = performance profiling and modification of code modules**

Section 6 — Flashcards

Cover the BACK row and recall it from the FRONT. All 45 cards are drawn from Chapter 1 of the CT-PT 2018 syllabus. Target: three clean passes before exam day.

FRONT	**1. In which standard's product quality model is performance efficiency categorized, and as what?**
BACK	**ISO 25010**, as a **non-functional quality characteristic**.
FRONT	**2. Name the three sub-characteristics of performance efficiency.**
BACK	**Time behavior · resource utilization · capacity.**
FRONT	**3. Which sub-characteristic is 'generally the most common performance testing objective'?**
BACK	**Time behavior** — the ability to **respond to user or system inputs within a specified time and under specified conditions**.
FRONT	**4. How wide can time-behavior measurements range?**
BACK	From the **"end-to-end" time** taken by the system to respond to user input, down to the **number of CPU cycles** required by a software component to execute a particular task.
FRONT	**5. When is resource utilization investigated?**
BACK	If the **availability of system resources is identified as a risk** — e.g., the **allocation of limited RAM**.
FRONT	**6. When is capacity investigated, and what does it evaluate?**
BACK	If **system behavior at the required capacity limits** (e.g., **numbers of users or volumes of data**) is identified as a risk. It evaluates the **suitability of the system architecture**.
FRONT	**7. Is scalability a fourth sub-characteristic?**
BACK	**No. All three** sub-characteristics **impact the ability of the SUT to scale**.
FRONT	**8. State the reproducibility principle in syllabus wording.**
BACK	**Statistically identical results (within a specified tolerance) must be obtained by repeating the tests on an unchanged system.**
FRONT	**9. Which three stakeholder groups does principle 1 name?**
BACK	**Users, system designers and operations staff.**
FRONT	**10. What must test results be, according to principle 3?**
BACK	**Both understandable and readily comparable to stakeholder expectations** — comparable, not matching.
FRONT	**11. Where may performance tests be conducted, according to principle 4?**
BACK	**Where resources allow**, on **complete or partial systems**, or **test environments representative of the production system**.
FRONT	**12. What does principle 5 require?**
BACK	The tests must be **practically affordable and executable within the timeframe set by the project**.
FRONT	**13. What form does performance testing often take, and to what end?**
BACK	**Experimentation** — enabling **measurement and analysis of specific system parameters**, conducted **iteratively in support of system analysis, design and implementation**, to enable **architectural decisions** and to **shape stakeholder expectations**.
FRONT	**14. Which architectures besides web-based does the syllabus name?**
BACK	**Classic client-server, distributed and embedded.**
FRONT	**15. Which two disciplines is performance testing often closely integrated with?**
BACK	**Usability engineering** and **performance engineering**.

FRONT	**16. Define performance testing (the umbrella term).**
BACK	An **umbrella term** including **any kind of testing focused on performance (responsiveness) of the system or component under different volumes of load**.
FRONT	**17. Define load testing.**
BACK	Focuses on the ability of a system to handle **increasing levels of anticipated realistic loads** resulting from transaction requests generated by **controlled numbers of concurrent users or processes**.
FRONT	**18. Define stress testing — both halves.**
BACK	Focuses on handling **peak loads at or beyond the limits of anticipated or specified workloads**; **also** used to evaluate the ability to handle **reduced availability of resources** such as **accessible computing capacity, available bandwidth and memory**.
FRONT	**19. Define scalability testing.**
BACK	Focuses on the ability to meet **future efficiency requirements which may be beyond those currently required** — the ability to **grow** (more users, larger data) **without violating the currently specified performance requirements or failing**.
FRONT	**20. What becomes possible once the limits of scalability are known?**
BACK	**Threshold values can be set and monitored in production** to warn of problems about to arise, and the **production environment may be adjusted with appropriate amounts of hardware**.
FRONT	**21. Define spike testing.**
BACK	Focuses on the ability to **respond correctly to sudden bursts of peak loads and return afterwards to a steady state**.
FRONT	**22. Define endurance testing, and name the three resource problems it targets.**
BACK	Focuses on **stability over a time frame specific to the system's operational context**. It verifies there are no **resource capacity problems** — **memory leaks, database connections, thread pools** — that may **eventually degrade performance and/or cause failures at breaking points**.
FRONT	**23. Define concurrency testing, and state its stated difficulty.**
BACK	Focuses on the impact of situations where **specific actions occur simultaneously** (e.g., many users logging in at once). Concurrency issues are **notoriously difficult to find and reproduce**, particularly where testing has **little or no control, such as production**.
FRONT	**24. Define capacity testing.**
BACK	Determines **how many users and/or transactions a given system will support and still meet the stated performance objectives** — objectives may also cover **data volumes resulting from the transactions**.
FRONT	**25. Capacity versus scalability in one line.**
BACK	**Capacity measures today's ceiling; scalability asks whether the ceiling can be raised.**
FRONT	**26. Which two testing types does §1.3 name?**
BACK	**Static testing** and **dynamic testing**.
FRONT	**27. Why is static testing often more important for performance testing than for functional suitability testing?**
BACK	Because **so many critical performance defects are introduced in the architecture and design of the system**.
FRONT	**28. Name two sources of those architecture and design defects.**
BACK	**Misunderstandings or a lack of knowledge by the designers and architects**; and **requirements that did not adequately capture response time, throughput or resource utilization targets, the expected load and usage, or the constraints**.
FRONT	**29. List the four static testing activities for performance.**
BACK	Reviews of **requirements** (performance aspects and risks) · **database schemas, entity-relationship diagrams, metadata, stored procedures and queries** · the **system and network architecture** · **critical segments of the system code** (e.g.,

	complex algorithms).
FRONT	**30. What dynamic performance testing happens during unit testing?**
BACK	Using **profiling information to determine potential bottlenecks** and **dynamic analysis to evaluate resource utilization**.
FRONT	**31. What happens at component integration testing?**
BACK	Testing across **key use cases and workflows**, especially when **integrating different use case features** or **integrating with the "backbone" structure of a workflow**.
FRONT	**32. What happens at system testing?**
BACK	Testing of **overall end-to-end behaviors under various load conditions**.
FRONT	**33. What happens at system integration testing, and who is the 'user'?**
BACK	Testing of **data flows and workflows across key inter-system interfaces**. It is **not uncommon for the "user" to be another system or machine** (e.g., sensor inputs).
FRONT	**34. What is the purpose of performance testing at acceptance testing?**
BACK	To **build user, customer and operator confidence** and to **fine tune the system under real world conditions** — **generally not to find performance defects**.
FRONT	**35. What is critical at higher test levels?**
BACK	The use of **realistic environments, data and loads** — critical for accurate results.
FRONT	**36. What should Agile and iterative-incremental teams do?**
BACK	**Incorporate static and dynamic performance testing into early iterations** rather than waiting for final iterations.
FRONT	**37. What is the advice on simulators for custom or new hardware?**
BACK	Early dynamic tests **can use simulators**, but **start testing on the actual hardware as soon as possible** — simulators **often do not adequately capture resource constraints and performance-related behaviors**.
FRONT	**38. Name the three ways a performance test load differs from functional test data.**
BACK	It must represent **many user inputs, not just one**; it **may require dedicated hardware and tools**; and its generation **depends on the absence of any functional defects in the SUT**.
FRONT	**39. What is 'a key success factor when conducting performance tests'?**
BACK	**The efficient and reliable generation of a specified load.**
FRONT	**40. Name the four load generation options.**
BACK	Via the **user interface** · using **crowds** · via the **API** · using **captured communication protocols**.
FRONT	**41. Why is UI-based load generation risky beyond small numbers?**
BACK	It **rapidly becomes impractical as user numbers increase**, and the **stability of the UI is a critical dependency** — **frequent changes impact repeatability and may significantly affect maintenance costs**.
FRONT	**42. When is API-based load generation the right answer?**
BACK	When the **UI is likely to change** but the **transactions must be processed as if they were created through the UI**. It is **less sensitive to UI changes** and allows **more users to be simulated** than UI input.
FRONT	**43. How does the API level relate to the communications protocol level?**
BACK	**The communications protocol level is one level below the API level.**
FRONT	**44. List the four common performance efficiency failure modes.**
BACK	**Slow response under all load levels · slow response under moderate-to-heavy load levels · degraded response over time · inadequate or graceless error handling under heavy or over-limit load.**

FRONT	**45. A memory leak produces which failure mode, and which other causes share it?**
BACK	**Degraded response over time** — shared with **disk fragmentation, increasing network load over time, growth of the file repository and unexpected database growth**.

Section 7 — Revision Notes

7.1 One-page chapter summary

Chapter 1 — Basic Concepts

Performance efficiency is an **ISO 25010 non-functional product quality characteristic** with three sub-characteristics — **time behavior** (the most common objective, measured from **end-to-end response** down to **CPU cycles**), **resource utilization** (investigated when **resource availability is a risk**, e.g. **limited RAM**) and **capacity** (investigated when **behavior at capacity limits** is a risk, evaluating the **suitability of the system architecture**). **All three impact the ability of the SUT to scale.** Five principles govern the work: tests **aligned to the expectations of users, system designers and operations staff**; **reproducible**, giving **statistically identical results within a specified tolerance on an unchanged system**; **understandable and readily comparable to stakeholder expectations**; conducted **where resources allow on complete or partial systems or representative environments**; and **practically affordable within the project timeframe**. Performance testing is **experimentation**, run **iteratively** to inform **architectural decisions** and **shape stakeholder expectations**, and it applies well beyond the web — to **client-server, distributed and embedded** systems. **Eight types** are defined: **performance testing** (umbrella) · **load** (**anticipated realistic loads to the expected maximum**) · **stress** (**at or beyond the limits**, plus **reduced resources**) · **scalability** (**future efficiency requirements**, then **production thresholds and hardware adjustment**) · **spike** (**sudden bursts, then back to steady state**) · **endurance** (**stability over time**; **memory leaks, database connections, thread pools**) · **concurrency** (**simultaneous actions**; **notoriously hard to reproduce**) · **capacity** (**how many, while still meeting objectives**). **Static testing matters more here than in functional testing**, because **critical performance defects come from architecture and design** — hence four review activities covering **requirements, database design, system and network architecture, and critical code**. **Dynamic testing starts as soon as possible**, with one opportunity per level: **unit = profiling and resource utilization**, **component integration = use cases and workflows**, **system = end-to-end under load**, **system integration = inter-system data flows** (where the user may be **another machine**), **acceptance = confidence and fine tuning, not defect finding**. Load must be **modeled, generated and submitted**; it differs from functional data by needing **many inputs**, possibly **dedicated hardware and tools**, and **a functionally defect-free SUT**. Four generation options trade scale against realism — **UI · crowds · API · captured protocols** — and **the protocol level sits one level below the API level**. Finally, four **failure modes** are discriminated purely by **when** they appear: **always** (design, latency, background load) · **only under moderate-to-heavy load** (resource saturation) · **progressively over time** (**memory leaks**, fragmentation, repository and database growth) · **only in the error handling** (**insufficient pools, undersized queues and stacks, too rapid time-outs**).

7.2 Key definitions to memorize

- **Performance efficiency** — the **ISO 25010 non-functional product quality characteristic** comprising **time behavior, resource utilization and capacity**.
- **Load testing** — **increasing levels of anticipated realistic loads** from **controlled numbers of concurrent users or processes**.
- **Stress testing** — **peak loads at or beyond the limits of anticipated or specified workloads**, and **reduced availability of resources**.
- **Scalability testing** — meeting **future efficiency requirements beyond those currently required**, without **violating the currently specified performance requirements or failing**.
- **Spike testing** — responding correctly to **sudden bursts of peak loads** and **returning afterwards to a steady state**.
- **Endurance testing** — **stability over a time frame specific to the system's operational context**; no **resource capacity problems** causing **failures at breaking points**.
- **Concurrency testing** — the impact of **specific actions occurring simultaneously**.
- **Capacity testing** — **how many users and/or transactions** the system supports **while still meeting the stated performance objectives**.
- **Static testing (for performance)** — reviews of **requirements, database design artefacts, system and network architecture, and critical code segments**.
- **Load generation** — the **modeling, generation and submission of representative system loads** to the SUT.

7.3 High-priority concepts (study first)

Priority	Concept	LO	Marks
★★★	The eight types, each with its signature phrase — especially the **load / stress / scalability / spike** quartet	PTFL-1.2.1	**2**
★★★	Which dynamic activity belongs at which of the five test levels	PTFL-1.3.1	1
★★★	The four failure modes discriminated by **when** they appear, plus their causes	PTFL-1.5.1	1
★★	The four load generation options; **API vs protocol level**; crowds use the **real UI**	PTFL-1.4.1	1
★★	The five principles, with their exact qualifiers — especially **reproducibility**	PTFL-1.1.1	1
★★	The three sub-characteristics and the fact that **scalability is not one of them**	PTFL-1.1.1	—
★	The four static review activities	PTFL-1.3.1	—
★	The six specific failure examples and their one-line diagnoses	PTFL-1.5.1	—

7.4 Last-minute revision sheet

- **6 of 40 questions · 6 of 40 points · K1 = 1, K2 = 5 · all objectives guaranteed · 60 minutes of teaching.** Sample Exam questions **#1–#6**. **PTFL-1.2 is the only two-mark objective.**
- **Performance efficiency = ISO 25010, non-functional.** Sub-characteristics: **time behavior · resource utilization · capacity**. **All three impact scalability.**
- **Principles:** aligned to **users / system designers / operations staff · reproducible (statistically identical, specified tolerance, unchanged system) · understandable and comparable · complete or partial systems or representative environments, where resources allow · affordable within the project timeframe**.
- **Types:** load = **expected maximum** · stress = **at or beyond**, plus **reduced resources** · scalability = **future** · spike = **sudden burst, return to steady state** · endurance = **over time**, leaks and pools · concurrency = **simultaneously** · capacity = **how many, still meeting objectives** · performance testing = **umbrella**.
- **Static:** requirements · **database schemas, ERDs, metadata, stored procedures, queries · system and network architecture · critical code segments**. It matters **more** here than in functional testing.
- **Dynamic ladder:** unit = **profiling, resource utilization** · component integration = **use cases, workflows, backbone** · system = **end-to-end under load** · system integration = **inter-system data flows, user may be a machine** · acceptance = **confidence and fine tuning, not defect finding**.
- **Load generation:** UI (**few users, most representative end-to-end, UI stability is a critical dependency**) · crowds (**real testers, many devices, less reproducible and precise, complex to organize**) · API (**UI likely to change, transactions as if via UI, more users than UI**) · captured protocols (**very large numbers, repeatable and reliable**). **Protocol is one level below API.**
- **Failure modes by timing:** always = **design, latency, background load** · under load = **resource saturation** · over time = **memory leak, fragmentation, repository and database growth** · error handling = **pools, queues, stacks, time-outs**.
- **Agile:** static and dynamic performance testing belong in **early iterations**. **Simulators early, real hardware as soon as possible.**

7.5 Memorization checklist

6. I can name the five learning objectives and say which one is worth **two** marks and which one is **K1**.
7. I can name the standard, the model and the characteristic type for performance efficiency.
8. I can define all three sub-characteristics and explain why scalability is not a fourth.
9. I can recite the five principles with their qualifiers.
10. I can define all eight types in one line each and identify each from a description.
11. I can reproduce the four-option block the official exam uses for load and spike.

12. I can separate **capacity** from **scalability**, and **spike** from **stress**, and **concurrency** from both.
13. I can list the four static review activities.
14. I can state the dynamic performance testing opportunity at each of the five test levels.
15. I can state why acceptance-level performance testing is **not** for finding defects.
16. I can name the four load generation options with one advantage and one limitation each.
17. I can state that the **communications protocol level is one level below the API level**.
18. I can list the four failure modes and give at least two causes for each.
19. I can map a **memory leak** to its failure mode without hesitating.
20. I can give the syllabus diagnosis for at least four of the six specific failure examples.

7.6 Exam-day quick review

60-second mental checklist before answering any Chapter 1 question

1) Which LO is this? (principles · types · static-dynamic · load generation · failure modes) **2)** If it is a **type** question, find the two decisive words: **how big** is the load and **how fast** does it change. **3)** If it is a **level** question, place the activity on the ladder **unit → component integration → system → system integration → acceptance**. **4)** If it is a **load generation** question, ask **how many users** and **is the UI stable** — and remember crowds use the **real UI** and protocol sits **below** API. **5)** If it is a **failure mode** question, ask **when does it hurt**: always · only under load · progressively · only in the error handling. **6)** Check the qualifiers — **comparable to** not matching, **representative of** production not production itself, **statistically identical within a tolerance** not bit-identical. **7)** If two options survive, choose the one that repeats **syllabus wording**.

Sources: ISTQB® Certified Tester Foundation Level Specialist Syllabus — Performance Testing, Version 2018 (9 December 2018), Chapter 1 and Keywords; ISTQB® CT-PT Sample Exam Questions and Sample Exam Answers, Version 2018 (9 December 2018); ISTQB® Exam Structure Tables v1.16 (CT-PT) and ISTQB® Exam Structures and Rules v1.2. All content is traceable to these references; no external knowledge was used.